Software Metrics and Complexity Analysis

Digital Wallet Management System (DWMS)

Group 5

1. Introduction

1.1 Purpose

This document measures the size, structure and complexity of the Digital Wallet Management System using standard software metrics. It answers three questions: how big is the system, how complex is its design and control flow, and where are the weak spots that should be fixed before coding starts.

1.2 Basis of measurement

DWMS is at the design stage and has no source code yet. Every metric below is therefore a **design-level metric** computed from the project artifacts, not from code, except the Halstead example in Section 6.5, which analyses a small illustrative code sample. Code-level metrics (lines of code, Maintainability Index, LCOM, test coverage) are planned for the implementation phase.

Artifact	Used for
SRS v1.0 (Sept 2026)	Requirement counts, function point estimate, quality targets
Class diagram and object diagram	Chidamber–Kemerer (CK) object-oriented metrics
Module table M1–M6 and decomposition diagram	Fan-in, fan-out, instability, coupling and cohesion
Activity diagram (4 swimlanes)	Cyclomatic complexity of the user flow
Sequence diagram (payment flow)	Cyclomatic complexity of the payment flow
Transaction state machine (INITIATED to SETTLED)	Cyclomatic complexity of the state logic

2. Requirements Metrics

2.1 Requirement counts (from the SRS)

Feature	Priority	Functional requirements
4.1 Registration and Authentication	High	6
4.2 Wallet Management	High	6
4.3 Fund Transfer and Payments	High	6
4.4 Transaction History and Reporting	Medium	4
4.5 Admin and Support	Medium	4
Total functional requirements		**26**

Non-functional category	Count
Performance (PERF-1 to 4)	4

Safety (SAFE-1 to 3)	3
Security (SEC-1 to 6)	6
Business rules (BR-1 to 4)	4
Quality attributes (reliability, usability, maintainability, portability, scalability)	5
Total non-functional requirements	**22**

Derived ratios

- Total requirements = 26 + 22 = **48**.
- Functional to non-functional ratio = 26 : 22, about 1.2 : 1. This is a security- and constraint-heavy spec, which is expected for a financial product.
- 18 of 26 functional requirements (69%) belong to High-priority features. Authentication, wallet and payments carry most of the delivery risk.
- Open items: 2 To-Be-Determined entries (payment gateway partners and final transaction limits). Requirements stability index is therefore high but not 100%.

2.2 Measurable quality targets in the SRS

Attribute	Target	What it means
Availability	99.5% uptime	At most about 43.8 hours of downtime per year (about 3.65 hours per month)
UI response	2 s	PERF-1
Payment confirmation	5 s	PERF-2
Concurrency	1,000 users	PERF-3
History query (30 days)	1 s	PERF-4
First-time usability	Register and top up within 10 min	Usability test criterion

3. Size Estimation

3.1 Function Point analysis

Function points were counted from the SRS using IFPUG average weights (EI 4, EO 5, EQ 4, ILF 10, EIF 7).

Type	Items identified	Count	Weight	Points
External Inputs (EI)	Register, login with OTP, password reset, add money, withdraw, link bank account, make payment or transfer, suspend account, resolve dispute	9	4	36
External Outputs (EO)	Statement (PDF/CSV), notification (SMS/email/push), admin dashboard metrics, payment receipt	4	5	20
External Inquiries (EQ)	View balance, view and filter history, view transaction status, search users	4	4	16
Internal Logical	User, Wallet, Transaction, BankAccount, Payment,	6	10	60

Files (ILF)	Admin			
External Interface Files (EIF)	Bank/payment gateway, SMS/email gateway, KYC service	3	7	21
Unadjusted FP (UFP)				**153**

Value Adjustment Factor(VAF) The 14 general system characteristics were rated 0 to 5:

Characteristic	Rating	Characteristic	Rating
Data communications	5	End-user efficiency	4
Distributed data processing	3	Online update	5
Performance	4	Complex processing	3
Heavily used configuration	2	Reusability	2
Transaction rate	4	Installation ease	2
Online data entry	5	Operational ease	2
Multiple sites	1	Facilitate change	3

Total Degree of Influence = 45, so VAF = 0.65 + 0.01 × 45 = **1.10**.

Adjusted FP = 153 × 1.10 ≈ 168 FP.

3.2 Effort, schedule and team size (COCOMO)

Assumptions: TypeScript/Node.js at about 50 lines of code per FP, giving 168 × 50 = **8,400 LOC ≈ 8.4 KLOC**. The Basic COCOMO *semi-detached* mode is used because the design mixes familiar web technology (React Native, Express) with less familiar parts (double-entry ledger, Redis locks, Kafka, ISO 8583).

Quantity	Formula	Result
Effort	3.0 × (KLOC)^1.12	**≈ 32.5 person-months**
Duration	2.5 × (Effort)^0.35	**≈ 8.5 months**
Average staffing	Effort / Duration	**≈ 3.8 persons**

The average staffing of about 4 matches the size of Group 5. These are order-of-magnitude figures for a professional team; for the academic submission they show that a full implementation of the six-tier design is larger than a semester project, which supports building a vertical slice first (login, OTP, balance check, PIN, payment, receipt).

4. Object-Oriented Design Metrics (Class Diagram)

The class diagram has 6 classes: Admin, User, BankAccount, Wallet, Payment, Transaction.

4.1 Per-class measurements

Class	Attributes	Methods (WMC, unit weight)	Associations (CBO)	DIT	NOC

Class	Attributes	Methods (WMC, unit weight)	Associations (CBO)	DIT	NOC
Admin	3	4	1 (User)	0	0
User	5	4	3 (Admin, BankAccount, Wallet)	0	0
BankAccount	4	3	1 (User)	0	0
Wallet	4	4	3 (User, Payment, Transaction)	0	0
Payment	4	3	2 (Wallet, Transaction)	0	0
Transaction	5	3	2 (Wallet, Payment)	0	0
Total / average	**25 / 4.2**	**21 / 3.5**	**12 / 2.0**	**0**	**0**

4.2 Interpretation

- **WMC (average 3.5)** is low and healthy. No class is a "god class"; the largest classes (Admin, User, Wallet) have only 4 methods.
- **CBO (average 2.0, maximum 3)** is low. The common guideline is to keep CBO at 5 or below.
- **DIT = 0 and NOC = 0** for every class: there is no inheritance. This is acceptable for a small domain model, but it means shared behaviour is duplicated (see below).
- **Relationship metrics:** 6 associations, 0 generalisations, 0 aggregations. Association density = 6 / 15 possible pairs = 0.40.

4.3 Design issues found while measuring

#	Issue	Why it matters	Suggested fix
1	`Admin` and `User` both define `login()` and share identity attributes (id, name, email)	Duplicated logic and attributes (DIT = 0)	Introduce a base `Account` class and make `Admin` and `User` subclasses; DIT becomes 1
2	Wallet, Payment and Transaction form a triangle (Wallet–Payment, Wallet–Transaction, Payment–Transaction)	Cyclic dependency raises coupling and complicates testing	Let Transaction reference Wallet only; link Payment to Transaction through one direction
3	`balance` and `amount` are typed `double`	Floating point rounding conflicts with the report's "zero balance drift" goal	Use a decimal or integer-paise type
4	`User.password : String`	Conflicts with Argon2id hashing in the design report	Rename to `passwordHash`
5	Typos in the diagram: `amout`, `unblovkUser`, `uppdateProfile`, `accountNumebr`	Inconsistent names become inconsistent code	Correct before implementation
6	`Payment` and `Transaction` are 1:1 with overlapping fields (amount, status, date)	Redundant data (normalisation concern)	Merge, or make Payment a type of Transaction

5. Architecture Metrics (Modules M1–M6)

5.1 Fan-in, fan-out and instability

Fan-out is the number of modules a module depends on (the "Depends on" column of the module table). Fan-in is the number of modules that depend on it. Instability I = Fan-out / (Fan-in + Fan-out); 0 is maximally stable, 1 is maximally unstable.

Module	**Fan-in**	**Fan-out**	**Instability I**	**Fan-out²**
M1 Client Presentation	1	0	0.00	0
M2 API Gateway and Edge	1	1	0.50	1
M3 Proxy / FSM Orchestrator	3	1	0.25	1
M4 Core Double-Entry Ledger	2	1	0.33	1
M5 Banking and Gateway Adapter	1	2	0.67	4
M6 Async Event and Notification Hub	0	3	1.00	9
Total	**8**	**8**	avg 0.46	**16**

The totals match (8 dependency edges), which is a consistency check on the table. Dependency density = 8 edges out of 15 possible in a six-node acyclic graph = **0.53**, and the module graph is acyclic.

5.2 Interpretation

- **M3 is the architectural hub** (fan-in 3). Everything in the money path runs through it, so it is the highest-risk module for change and for performance. It deserves the strongest unit and integration test coverage.
- **M6 (I = 1.0) is the most unstable**, which is correct for a leaf consumer of events. Because it is reached through Kafka publish-subscribe, changes to it do not ripple back.
- **M4 (the ledger) has I = 0.33**, so it is relatively stable. This suits the module that must never change behaviour casually.
- Average fan-out of 1.33 per module is low, consistent with the report's low-coupling claim.

5.3 Coupling and cohesion scores

Using Constantine's scales (coupling: data 1, stamp 2, control 3; cohesion: functional 7, sequential 6, communicational 5), the interactions already analysed in the Design Analysis Report score as follows.

Module pair	Coupling type	Score
M2 – M3	Data	1
M3 – M4	Stamp	2
M3 – M6	Control	3
Average coupling		**2.0 (low)**

Module	Cohesion type	Score
M4 Ledger	Functional	7

Module	Cohesion type	Score
M3 Transaction Engine	Sequential	6
M5 Banking Adapter	Communicational	5
Average cohesion		**6.0 (high)**

A design target of average coupling 3 or below and average cohesion 5 or above is met. The weakest point is M3, whose fee rules and state transitions are bundled together; the report's own suggestion to separate fee calculation from state transitions would raise it to functional cohesion.

Naming inconsistency. M3 is called "Client Presentation Layer Proxy" in the module table, "Proxy & FSM Orchestrator" in the decomposition diagram, and "Transaction Routing Engine" in the coupling table. One name should be used everywhere so the metrics can be traced to a single module.

6. Complexity Metrics (Cyclomatic and Halstead)

Formula: V(G) = number of decision nodes + 1 (equivalently E − N + 2). Thresholds: 1–10 simple, 11–20 moderate, 21–50 complex, above 50 untestable.

6.1 Activity diagram (user flow)

Decision nodes: (1) Login valid?, (2) Valid OTP?, (3) Sufficient balance?, (4) Payment method (UPI or Card). The fork and join around *Create Transaction Record* and *Update Wallet Balance* model concurrency, not a decision, and the OTP timer is an event, so neither adds to V(G).

V(G) = 4 + 1 = 5. The five independent paths are:

1. Login invalid, return to Login.
2. Login valid, OTP invalid, re-enter OTP.
3. OTP valid, balance insufficient, Add Money through the bank, validate, process, update balance.
4. OTP valid, balance sufficient, pay by UPI.
5. OTP valid, balance sufficient, pay by Card.

6.2 Sequence diagram (payment flow)

One alt frame (balance check) gives **V(G) = 2**. Two outcomes the report itself mentions are not drawn: a wrong PIN (lockout after repeated attempts) and a bank decline or timeout. Modelling them would raise the true value to **4**.

6.3 Transaction state machine

States: INITIATED, AUTHORIZED, PROCESSING, SETTLED, FAILED, REVERSED, plus the initial and final nodes (N = 8). Transitions: 10 (E = 10).

V(G) = 10 − 8 + 2 = 4. The four independent paths are Initiated → Failed; Initiated → Authorized → Processing → Failed; Initiated → Authorized → Processing → Settled; and the same path continuing Settled → Reversed.

6.4 Summary

Model	Decisions	V(G)	Rating

Model	Decisions	V(G)	Rating
Activity diagram	4	5	Simple, low risk
Sequence diagram (as drawn)	1	2	Simple (incomplete)
Sequence diagram (with PIN failure and bank decline)	3	4	Simple
Transaction state machine	–	4	Simple

All flows are well under the threshold of 10, so no flow needs to be split for complexity. The minimum number of **basis-path test cases** is 5 + 4 = **9**, before adding requirement-based, boundary and security tests.

6.5 Halstead metrics (code sample)

Halstead metrics measure code by counting its operators and operands. No DWMS source code exists yet, so this section applies the method to a seven-line sample of the wallet balance check (the "Check Available Balance" step in the sequence diagram). The same procedure will be run on real modules once they are written.

```
function debit(balance, amount) {
  if (balance < amount) {
    return 0;
  }
  balance = balance - amount;
  return balance;
}
```

Counting rules. Keywords, symbols, each bracket pair, commas and semicolons are operators. Identifiers (including the function name) and literals are operands.

Operator	Count	Operand	Count
function	1	balance	5
( )	2	amount	3
,	1	debit	1
{ }	2	0	1
if	1		
<	1		
return	2		
=	1		
-	1		
;	3		
n1 = 10 distinct, N1 = 15 total		**n2 = 4 distinct, N2 = 10 total**	

Metric	Formula	Result
Vocabulary (n)	n1 + n2	14
Length (N)	N1 + N2	25
Calculated length (N^)	n1 log2 n1 + n2 log2 n2	41.2
Volume (V)	N × log2 n	95.2
Difficulty (D)	(n1 / 2) × (N2 / n2)	12.5
Effort (E)	D × V	about 1,190
Time to program (T)	E / 18 seconds	about 66 s
Estimated bugs (B)	V / 3000	0.03

Reading the result. The volume of 95 is tiny, far below the 1,000 per function that review tools commonly flag. Difficulty is 12.5 because `balance` appears 5 times but the function has only 4 distinct operands, so each operand is reused heavily. The same snippet has one decision, so its cyclomatic complexity is V(G) = 2, matching the alt frame in the sequence diagram (Section 6.2).

Halstead values on one short function say little alone; they become useful when compared across functions. Counting conventions differ slightly between tools (for example how brackets are counted), so all DWMS modules should be measured with the same tool.

7. Requirements-to-Design Coverage

Each of the 26 functional requirements was checked against the design artifacts.

Status	Count	Requirements
Fully covered	15	4.1.2, 4.1.4, 4.2.2, 4.2.3, 4.2.5, 4.2.6, 4.3.1, 4.3.2, 4.3.4, 4.3.5, 4.3.6, 4.4.1, 4.4.4, 4.5.2, 4.5.4
Partially covered	5	4.1.1 (registration only as a class method), 4.1.5 (lockout only mentioned as a risk), 4.2.1 (wallet class, no creation flow), 4.3.3 (QR only in M1), 4.5.3 (dispute only as the REVERSED state)
Not covered	6	4.1.3 password rules, 4.1.6 password reset, 4.2.4 transaction limits, 4.4.2 history filters, 4.4.3 statement export, 4.5.1 admin dashboard

Coverage = (15 + 0.5 × 5) / 26 ≈ 67% (58% counting only fully covered requirements).

The uncovered items are mostly Medium-priority features, but **4.2.4 (limits)** supports regulatory rules BR-2 and should be designed early. Suggested additions: a registration and password-reset sequence diagram, a limits-check step in the payment flow, and a history and statement use case.

8. Findings and Recommendations

1. **Close the coverage gap.** Add design for password rules and reset, transaction limits, history filters and statements, and the admin dashboard (Section 7).
2. **Complete the payment sequence diagram** with wrong-PIN lockout and bank decline or timeout paths, matching the FSM transition to FAILED.
3. **Introduce an `Account` base class** for Admin and User to remove duplicated login and identity fields.
4. **Break the Wallet–Payment–Transaction triangle** and decide whether Payment and Transaction should be merged.
5. **Use a decimal or integer money type** instead of `double`, and rename `password` to `passwordHash`.
6. **Protect M3.** It has the highest fan-in; give it the strictest tests and consider splitting fee rules from state transitions.
7. **Standardise names** (M3 title, class diagram typos) so metrics and diagrams agree.
8. **Plan a vertical slice first** (login, OTP, balance, PIN, payment, receipt), given the effort estimate in Section 3.2.

9. Assumptions and Limitations

- All metrics are computed from design documents. Code-level values (LOC, LCOM, Maintainability Index, defect density) will differ and must be re-measured once code exists.
- Function point weights are all "average" and the 14 characteristic ratings are the team's judgement; a different rating changes VAF within roughly 0.65 to 1.35.
- The 50 LOC per FP conversion and COCOMO constants are industry approximations and give an estimate, not a commitment.
- Fan-in and fan-out follow the "Depends on" column of the module table exactly as written. Two-way arrows in the decomposition diagram (M3–M4, M4–M5) are treated as request and response, not as cyclic dependencies.
- Coverage in Section 7 reflects what the current diagrams show; some requirements may be handled in the implementation even though no diagram draws them.